Planners Exploit Deterministic Error: Dithered Low-Bit World Models

Author Names Omitted for Anonymous Review. Paper-ID [add your ID here]

***Abstract*—Sampling-based planners such as MPPI and CEM query a learned world model thousands of times per control step, so low-bit world models are an obvious way to make model-based control cheaper on a robot. We find that at aggressive bit widths planning can collapse, by up to 100 points of success, while the world model's latent one-step prediction error stays no larger than that of a stochastic rounding scheme that plans well. We trace the damage to the structure of the rounding error rather than its size. Deterministic round-to-nearest leaves an error that persists across planner iterations, and an iterating planner drifts toward the candidates that this error favors; in TD-MPC2, holding the same dither fixed within a step or an episode brings the drift back. Dither shared by all candidates and redrawn at every iteration removes most of the damage at the same bit width and without training: at least half of it in all 19 affected Meta-World cells of TD-MPC2 (median 95%), in 4 of 5 DINO-WM cells, and in all 8 affected tasks of a 317M-parameter multi-task TD-MPC2. Replayed as CUDA graphs, the planning rollouts of a W8A4 integer world model run 1.9–3.3× faster than FP32 on an Ampere GPU, and shared dither adds at most 2.1% latency. Block-scaled 4-bit floating-point formats greatly reduce the damage (NVFP4 damaged 2 of 12 tasks), which locates the failure in coarse grids with a shared scale. Hypotheses, thresholds and analysis code were frozen before the data that test them, and the hypotheses that failed are reported.**

I. Introduction

A sampling-based planner evaluates its world model candidates × iterations × horizon times per decision: $512 \times 6 \times 3$ forward calls for the MPPI planner of TD-MPC2 [6] and $300 \times 30 \times 5$ for the CEM planner of DINO-WM on PushT [25]. The world model's arithmetic therefore dominates the cost of each control step, and low-bit integer arithmetic is the most direct way to cut it. The cut is real: replayed as a CUDA graph, the planning rollouts of a W8A4 TD-MPC2 world model run 1.9–3.3× faster than FP32 on an Ampere GPU, from 5M to 317M parameters (Sec. VI). Yet quantized world models break planning in ways that bit width and offline prediction error do not explain [3, 22]. At aggressive bit widths we see planning collapse, by up to 100 points of success, while in most affected settings the world model's latent one-step error is no larger than that of a stochastic rounding scheme that plans well.

We show that what matters is the structure of the rounding error, not its size. Round-to-nearest (det) is a fixed function of its input, so the error it adds to the predicted return of a given action sequence is the same in every planner iteration. An iterating planner keeps the candidates that this error favors and moves its sampling distribution toward them. Dither, i.e. stochastic rounding $\Delta\lfloor x/\Delta + u \rfloor$ with $u \sim U[0, 1)$, is unbiased; its error is somewhat larger than that of round-to-nearest, but it can be redrawn. Our fix, sh-iter, shares one dither draw across all candidates and redraws it at every forward call, hence at every iteration. Sharing keeps the comparison between candidates fair within an iteration, as with common random numbers [7], and redrawing denies the planner an error it can exploit across iterations.

The ingredients are classical: the optimizer's curse [16], the bias of sample-average approximation [8], common random numbers, and unbiased stochastic rounding for iterative computation [4, 2]. What is new is their application to compressed world models inside closed-loop planners, a control that separates the two properties of the error, and a training-free fix whose effect we measure across two model families, two planners and two model sizes. Our contributions are:

1) **Phenomenon** (Sec. IV). At the same bit width, det causes planning damage that the latent one-step error does not reveal: in 37 of 44 affected cells its error is no larger than that of sh-iter.
2) **Mechanism** (Sec. V). The planner exploits error that persists across its iterations. In TD-MPC2, positive controls that hold the same dither fixed within a step or an episode reproduce 70–130% of det's drift, and the benefit of sh-iter grows with the number of iterations.
3) **Prescription** (Sec. IV). sh-iter removes at least half of the damage in all 19 affected Meta-World cells of TD-MPC2, in 4 of 5 DINO-WM cells and in all 8 affected tasks of a 317M multi-task TD-MPC2, and it is never harmful there.
4) **Deployment** (Sec. VI). Integer low-bit world models are fast when executed as CUDA graphs, dither costs at most 2.1% latency, and block-scaled FP4 formats mark the boundary of the failure.

All hypotheses were internally pre-registered: hypotheses, thresholds, seeds and analysis code were frozen in time-stamped, hashed documents before the data that test them. Hypotheses added after a stage are marked as such, and failed hypotheses are reported (Table I). Sec. VII lists the extensions we propose for Stage 2.

II. Related Work

Quantized world models and policies. Post-training quantization of DINO-WM makes the planning objective and task success diverge at aggressive bit widths [3]; offline action deviation predicts the closed-loop loss of quantized world action models [22]; and closed-loop benchmarks of quantized vision-language-action models report temporally accumulating error [19, 18]. These studies diagnose or allocate bits; none changes the rounding rule, and none separates how the error is

shared and how long it persists. **Prediction error versus planning quality.** Data-averaged prediction error does not track planning quality, while decision-relevant error does [23, 20]. Closest to us, Min et al. [9] split end-point error across a candidate pool into a pool-common part and an action-relative part and repair each with an oracle. We add the axis of persistence across optimizer iterations, a real error source (rounding), and a deployable fix. **Model exploitation.** Planners exploit model error [1]; World Models injected temperature noise into imagined rollouts so that the controller could not exploit them [5]; in CEM the world model also acts as a proposal mechanism [11]. We study compression error, whose size is fixed by the bit width, so that only its structure can be changed. **Dither and stochastic rounding.** Dither decorrelates quantization error from the signal [12, 15], and unbiased stochastic rounding prevents bias from accumulating in long computations [4, 2]. These results concern accumulation; the selection bias that an optimizer's argmax adds is the part we study.

III. Setup

Models and planners. TD-MPC2 [6]: the official 5M single-task checkpoints (three seeds) on 8 Meta-World tasks [24] (success rate) and 4 DMC tasks [17] (return), planned with the default MPPI [21] (512 samples including 24 policy trajectories, 64 elites, 6 iterations, horizon 3); and the official 317M multi-task checkpoint (mt80) on the same 8 Meta-World tasks. DINO-WM [25]: the official PushT, Wall and PointMaze models with open-loop CEM [14] (300 samples, top 30, horizon 5, 30 iterations on PushT and 10 otherwise). All planner settings are the official defaults unless an experiment varies them.

Quantization. We quantize hidden activations: the four hidden layers of the dynamics and reward MLPs in TD-MPC2 (after Mish) and the six MLP outputs of the DINO-WM predictor (after GELU), each to k bits on a uniform grid $[\min, q_{99.99}]$ calibrated per layer on held-out data. Inputs, weights and all other modules stay in floating point except in the integer kernels of Sec. VI. Per task, a pre-registered rule picks the primary k on held-out episodes as the largest k at which det loses at least 20 points of success (20% of return on DMC), and we test $k-1$, k and $k+1$: 36 cells for TD-MPC2 at 5M and 9 for DINO-WM. For each 317M task we test the primary k (3–5 bits).

Rounding variants. All stochastic variants use the same unbiased dither and therefore the same per-entry error distribution; they differ only in how u is shared and how long it is kept. ind draws u independently per candidate at every forward call. sh-iter shares one draw per layer across all candidates and redraws it at every forward call. sh-step and sh-ep share the same draw but hold it fixed within a control step or an episode (they coincide in open-loop CEM, where we call them sh-plan). det+shift keeps round-to-nearest but shifts the shared grid by a fresh offset at every call.

Metrics and statistics. Every variant is paired with the unquantized model on the same episode seeds and planner noise (90 pairs per cell for TD-MPC2: three checkpoints $\times$ 30 seeds, or one 317M checkpoint $\times$ 90 seeds; 100 for DINO-WM). The damage D is the drop from that baseline in success points (relative return on DMC). A cell is *affected* if the det drop is real under a one-sided paired bootstrap (10^4 resamples) with Holm correction over the cells of a task. The fraction of damage removed by a variant v is red $= (D_{\mathsf{det}} - D_v)/D_{\mathsf{det}}$, and v is *harmful* if the 95% interval of $D_v - D_{\mathsf{det}}$ lies above 5 points. The latent one-step error is the relative error of the predicted next latent state for the first-iteration candidates at planning time.

Exploitation. Let $e(a)$ be the error of the quantized model's predicted return for candidate a relative to the unquantized model. The optimizer exploitation index OEI $= e(a_{\text{exec}}) - \bar{e}_{\text{first}}$ compares the executed plan with random first-iteration candidates and splits exactly into a *selection* term (the last iteration's pick against its population) and a *drift* term, the change in the population-mean error from the first to the last iteration. Negative drift means the planner moved its population toward candidates that the error favors.

Pre-registration and compute. The study ran in stages: environment checks, the main benchmark, mechanism and structure ablations, post-training baselines, integer kernels, and the extensions marked in Table I. Before each stage, its hypotheses, thresholds, seeds, sample sizes and analysis code were frozen in a design document whose hash was logged before any of the stage's data existed; operational choices made in smoke tests, such as kernel tiling, were recorded before the main runs. Analyses chosen after seeing data are labeled post hoc. The main study used about 147 GPU-hours on RTX A5000 GPUs, and the extensions used cloud A40, RTX 4090 and RTX 5090 GPUs. Code, design documents and their hashes will be released with the final paper.

IV. Phenomenon and Prescription

Damage that the latent error does not show. det costs up to 100 points (mw-button-press at 2 bits). In 29 of the 36 affected 5M cells its latent one-step error is no larger than that of sh-iter (Fig. 1b). On mw-pick-place at 3 bits, for example, the error is 0.22 for det and 0.39 for sh-iter, yet det loses 37.8 points and sh-iter 7.8. The 317M model shows the same pattern in all 8 affected tasks: det's latent error is 0.55–0.69 times that of sh-iter while det loses 12–99 points. The comparison is within a bit width; across bit widths one-step error and damage do rank together. Two other error measures do separate det: errors over three or five steps (PushT, 4 bits, five steps: 0.25 for det vs. 0.04 for sh-iter), and, observed after the fact, the one-step error of the predicted reward, which is larger under det in all 19 affected Meta-World cells and all 8 tasks at 317M. A deterministic error is correlated across units and adds up in a scalar output, while independent dither errors partly cancel; we have not tested this explanation and list it for Stage 2 (Sec. VII).

Prescription. sh-iter removes at least half of the damage in all 19 affected Meta-World cells (median 95%) and in 4 of 5 DINO-WM cells, and it is not harmful in any of them

TABLE I

PRE-REGISTERED HYPOTHESES AND OUTCOMES. ROWS MARKED † WERE PRE-REGISTERED AFTER THE FIRST STAGE ON NEW SEEDS; ‡ MARKS THE EXTENSION EXPERIMENTS RUN FOR THIS SUBMISSION. MECHANISM TASKS: MW-PUSH, MW-PICK-PLACE, CHEETAH-RUN, PUSHT.

	Hypothesis and test	Outcome
H1	det's latent one-step error $\leq$ sh-iter's in some affected cell of each family	holds: 29/36 cells (TD-MPC2 5M, DINO-WM); 317M‡ 8/8
H3a	sh-iter removes $\geq$50% of the damage in $\geq$75% of affected cells, none harmful	Meta-World 19/19 (median 95%), DINO-WM 4/5, 317M‡ 8/8; DMC fails (5/12)
DMC†	Spearman(persistent share of det's error, red of sh-iter) ≥ 0.5 over 12 tasks	holds: 0.83
H2a′†	det drift < 0 and drift(sh-iter)/drift(det) < 0.5	holds: 4/4 tasks (ratio ≤ 0.24)
H2c	Spearman(candidate count, $\lvert\text{OEI}_{\text{det}}\rvert$ and D_{det}) ≥ 0.8, elites fixed	holds: 3/4 tasks
H4a	Spearman(table size of fixed dither, its damage) ≤ -0.8	holds: 5/6 tasks (three exactly at -0.8)
H3b	sh-iter beats det on top of the best post-training quantization	TD-MPC2 holds (10/19); DINO-WM fails (1/3)
H5	Dither latency overhead $\leq$10% in a fused integer kernel	holds (max 6.9%); integer kernel 18% slower than FP32 (eager)
L1–L3‡	Integer rollout $\leq 0.8\times$ FP32 time and $\leq 0.9\times$ the faster of TF32/BF16 in some cell; dither $\leq$5% everywhere	all hold: 12/12, 10/12, max overhead 2.1%
F1–F3‡	FP4 formats: det damages $\geq$3 of 12 tasks; sh-iter removes $\geq$50%; det's error $\leq$ sh-iter's	NVFP4 fails F1 (2/12), MXFP4 holds (4/12); F2 holds on Meta-World (1 cell), fails on DMC (2/5); F3 holds (6/6)
H2a, H2b	Original OEI tests: OEI$\approx$0 under sh-iter; OEI predicts damage better than one-step error	fail (claims narrowed to drift)
H4b	Benefit of sharing grows with the density of near-ties	fails ($\rho = 0.03$)

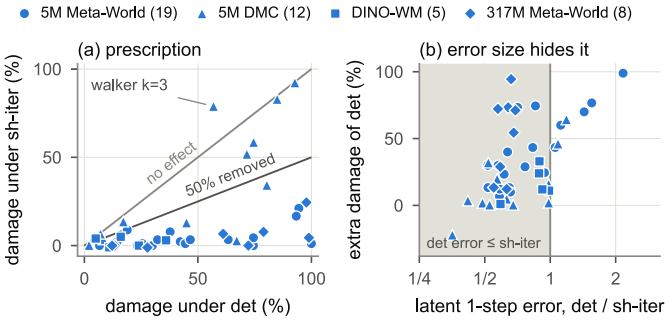


Fig. 1. Every affected cell. (a) Damage under sh-iter vs. det (success points on Meta-World and DINO-WM, relative return on DMC); below the dashed line sh-iter removes at least half of the damage. (b) det's extra damage against the ratio of the two latent one-step errors: most cells sit in the shaded region, where det's error is the smaller one.

(Fig. 1a, Table I). The weakest Meta-World cell keeps 47% of the damage (window-open, 4 bits), and the DINO-WM exception is PushT at 5 bits, where det's damage is only 5 points. On the 317M multi-task model, which needs no change to the method, sh-iter removes 75–104% of the damage in all 8 tasks (e.g. mw-peg-insert-side: 98.9 points lost under det, 4.4 under sh-iter), and the 95% interval of $D_{\text{sh-iter}} - D_{\text{det}}$ lies below zero in every task. The primary bit widths at 317M (3–5) are close to those at 5M (3–4), so the larger model is not more robust to rounding.

Where it helps less. On DMC, sh-iter removes half of the damage in only 5 of 12 cells and is harmful on walker at 3 bits (78.9% of return lost vs. 56.9% under det). A rule pre-registered after the first stage explains where it helps: the persistent share of det's favorable error (drift over OEI) is 0.33–0.52 on DMC and 0.54–0.73 on Meta-World, and across the 12 TD-MPC2 tasks it predicts the damage removed (Spearman 0.83, new seeds). The two DMC tasks with the smallest persistent share are the two where sh-iter fails.

V. MECHANISM

Drift. Under det the population-mean error drifts in the favorable direction in all four mechanism tasks at their primary bit widths. sh-iter keeps at most 24% of that drift (0.10, 0.24, 0.03 and -0.01 of it), and det+shift, which keeps round-to-nearest but moves the grid at every call, keeps at most 15%. The positive controls isolate persistence: holding the *same* shared dither fixed for a control step (sh-step) or an episode (sh-ep) brings back 70–130% of det's drift in the three TD-MPC2 tasks (Fig. 2a). Error size and sharing are equal across these variants; only persistence differs. On PushT the full-size fixed table did not reproduce the drift (ratio 0.01); there a fixed dither was exploited only when we made its table of offsets coarse, as below.

Iterations, candidates and table size. The fix needs an iterating planner. With one MPPI iteration sh-iter is no better than det; at 12 iterations its damage is 0–11% while det's stays at 43–47% and sh-step's at 18–47% (Fig. 2b). With the number of elites fixed, det's exploitation $\lvert\text{OEI}\rvert$ and damage grow with the number of candidates in three of four tasks, the dose-response expected from the optimizer's curse [16]. A fixed dither drawn from a table of N_{eff} independent offsets hurts less as the table grows (five of six tasks, three of them exactly at the threshold), and in an exploratory Wall cell (3 bits) a single offset per layer collapsed planning (+37 points).

Theory. For a planner that averages fresh unbiased noise, the expected drift is zero. A persistent error field instead pulls the population toward the minimizer of the smoothed model cost, the bias of sample-average approximation [8]. The selection and drift terms we derived matched 922k simulated MPPI and CEM runs (fitted slopes 0.92–1.05) and predicted, before the first stage was analyzed, that the OEI would be negative even under fresh noise, because the last iteration's

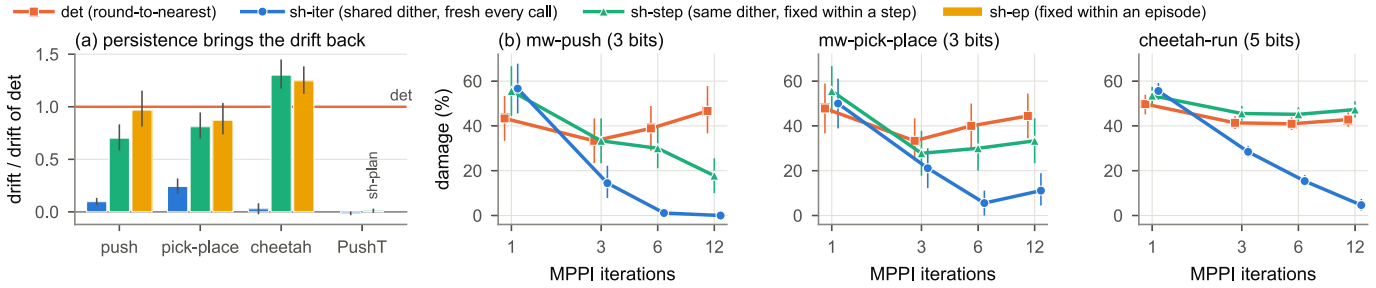


Fig. 2. Mechanism. (a) Favorable drift of the population-mean error kept by each variant, relative to det (TD-MPC2 at its primary bit width; PushT at 4 bits, where sh-step and sh-ep coincide as sh-plan); bars show 95% bootstrap intervals. (b) Damage vs. MPPI iterations (TD-MPC2, 90 paired episodes per point, 95% intervals). Fresh dither needs iterations to help; det does not improve with them.

pick benefits from the noise it was selected on (the winner's curse). This is why the pre-registered OEI tests H2a and H2b failed, and why our mechanism claim concerns the drift rather than the OEI.

VI. Deployment: Latency and Number Formats

Integer kernels. A fused Triton kernel (int8 weights, 4- or 8-bit activation codes, shared dither drawn on the GPU) reproduces the fake-quantization results in closed loop, and 8-bit activations cause no damage. Run eagerly at 5M parameters and 512 candidates, it was 18% slower than FP32: the step is bound by kernel launches, not arithmetic. To map where low bit widths pay off, we timed one control step's world-model rollouts (6 iterations $\times$ 3 horizon steps of dynamics and reward for N candidates, plus the policy trajectories) replayed as a single CUDA graph, as TD-MPC2's compiled planner does. The integer path uses int8 weights (per output channel) and 4-bit activation codes with int8 tensor-core GEMMs; the first layer, whose input $[z, a]$ is not quantized, stays in FP32. On an NVIDIA A40, across 5M–317M parameters and $N \in \{512, 2048, 8192\}$ (10 interleaved rounds per condition), the integer rollout is 1.9–3.3$\times$ faster than FP32 in all 12 cells and 1.19–1.48$\times$ faster than the faster of TF32 and BF16 in 10 of 12 (Fig. 3); shared dither changes latency by -0.2 to $+2.1$%, because one draw per layer and call serves every candidate. The gains are largest at 317M, the size at which the multi-task results of Sec. IV hold. The full planning step, run eagerly with the encoder, policy and eight Q-functions in FP32, gains less: 1.35$\times$ over FP32 and 1.05$\times$ over the fastest float format at 317M.

FP4 formats mark the boundary. Recent accelerators execute 4-bit floating point natively; on an RTX 5090, an NVFP4 GEMM at the 317M layer width is up to 5.3$\times$ faster than BF16. We therefore repeated the 5M experiment with the activations in NVFP4 [10] (blocks of 16 with an E4M3 scale) and MXFP4 [13] (blocks of 32 with a power-of-two scale), rounding each block to the E2M1 grid either to nearest (det) or stochastically (sh-iter). Block scaling greatly reduces the damage. det damaged only 2 of 12 tasks under NVFP4 (cheetah-run 9.9%, walker 1.7%), short of the pre-registered three, and 4 of 12 under MXFP4 (cheetah-run

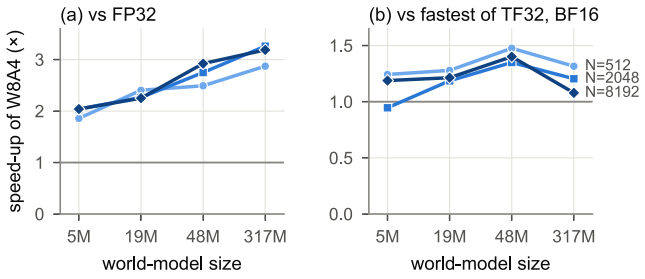


Fig. 3. Speed-up of the W8A4 integer world model over FP32 (a) and over the faster of TF32 and BF16 (b) for one control step of MPPI rollouts replayed as a CUDA graph (NVIDIA A40; N candidates, 6 iterations, horizon 3). BF16 uses autocast, which converts the weights at every call.

31.3%, window-open 10.0 points), against 71.6% on cheetah-run with a uniform 4-bit grid. Where damage remained, the signature was the same: det's latent one-step error was 0.48–0.63 times that of sh-iter in all six damaged cells, and sh-iter removed most of the damage in the one damaged Meta-World cell but in only 2 of 5 DMC cells, as with uniform grids. Fine block scaling shrinks the error; dither removes its persistence. The two remedies act on different axes, and the failure we study belongs to coarse grids with a shared scale.

VII. Limitations and Stage 2 Plan

Limitations. The prescription assumes an iterating planner, and on DMC it helps in fewer than half of the affected cells. On top of the best post-training quantization (mostly per-channel ranges), which already removes most of det's damage, the extra gain of sh-iter is borderline in TD-MPC2 (10 of 19 cells) and absent in DINO-WM (1 of 3). Our quantization-aware-training baseline was weak, so we draw no conclusion about training. The 317M results come from one checkpoint, the latency map from one GPU type with the first layer in FP32, and the FP4 results from fake quantization of the 5M models. We have no physical-robot experiments.

Proposed Extension Charter. Each item below would be pre-registered in the same way as Table I, with hypotheses, thresholds and analysis code frozen before its data.

1) *A diagnostic that sees the damage.* The latent one-step error misses det's damage, while the one-step reward

error separated det in every affected Meta-World cell (an observation made after the fact). We will test on new tasks and seeds whether the correlation of rounding error across hidden units, which a scalar output such as the reward sums up, predicts det's damage within a bit width, and whether a reward-error check run before deployment flags the damaging configurations.

2) *Integer kernels for DINO-WM.* We will port the integer path to the DINO-WM predictor and test whether closed-loop damage and repair match fake quantization in the affected cells, with the same latency budget for dither ($\leq$5%).
3) *The whole planning step.* We will time the complete MPPI step (encoder, policy, Q-ensemble, sampling and selection) captured as one CUDA graph, against FP32, TF32 and BF16 with pre-converted weights, and relate the speed-up to the control rate it allows.
4) *Embedded and FP4 hardware.* We will repeat the latency map on an NVIDIA Jetson module and, with fused quantization kernels, for NVFP4 on a Blackwell GPU, including the cost of drawing dither.

The negative results above (DMC, the combination with strong post-training quantization, planners with few iterations) stay in scope as boundaries to report, not as targets to tune away.

References

[1] Rinu Boney, Norman Di Palo, Mathias Berglund, Alexander Ilin, Juho Kannala, Antti Rasmus, and Harri Valpola. Regularizing trajectory optimization with denoising autoencoders. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[2] Matteo Croci, Massimiliano Fasi, Nicholas J. Higham, Theo Mary, and Mantas Mikaitis. Stochastic rounding: Implementation, error analysis and applications. *Royal Society Open Science*, 9(3):211631, 2022.

[3] Z. Fu et al. An empirical study of world model quantization. *arXiv preprint arXiv:2602.02110*, 2026.

[4] Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. Deep learning with limited numerical precision. In *International Conference on Machine Learning (ICML)*, pages 1737–1746, 2015.

[5] David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[6] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024.

[7] Nathan L. Kleinman, James C. Spall, and Daniel Q. Naiman. Simulation-based optimization with stochastic approximation using common random numbers. *Management Science*, 45(11):1570–1578, 1999.

[8] Anton J. Kleywegt, Alexander Shapiro, and Tito Homem-de Mello. The sample average approximation method for stochastic discrete optimization. *SIAM Journal on Optimization*, 12(2):479–502, 2002.

[9] R. Min et al. When low prediction error misleads planning. *arXiv preprint arXiv:2610.05550*, 2026.

[10] NVIDIA, Felix Abecassis, Anjulie Agrusa, et al. Pretraining large language models with NVFP4. *arXiv preprint arXiv:2509.25149*, 2025.

[11] Obst and Stolzenburg. In CEM, a world model is also a proposal mechanism. *arXiv preprint arXiv:2610.00921*, 2026.

[12] Lawrence G. Roberts. Picture coding using pseudo-random noise. *IRE Transactions on Information Theory*, 8(2):145–154, 1962.

[13] Bita Darvish Rouhani, Ritchie Zhao, Ankit More, et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.

[14] Reuven Y. Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999.

[15] Leonard Schuchman. Dither signals and their effect on quantization noise. *IEEE Transactions on Communication Technology*, 12(4):162–165, 1964.

[16] James E. Smith and Robert L. Winkler. The optimizer's curse: Skepticism and postdecision surprise in decision analysis. *Management Science*, 52(3):311–322, 2006.

[17] Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Riedmiller. DeepMind control suite. *arXiv preprint arXiv:1801.00690*, 2018.

[18] TODO. QVLA. *arXiv preprint arXiv:2602.03782*, 2026.

[19] TODO. VLAQuantBench. *arXiv preprint arXiv:2609.25376*, 2026.

[20] Wang, Fan, and Huang. Not all errors matter. *arXiv preprint arXiv:2609.32322*, 2026.

[21] Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews, James M. Rehg, Byron Boots, and Evangelos A. Theodorou. Information theoretic MPC for model-based reinforcement learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 1714–1721, 2017.

[22] J. Xu et al. Predict before you deploy. *arXiv preprint arXiv:2609.19441*, 2026.

[23] You et al. A control theory of predictability in latent world models. *arXiv preprint arXiv:2607.10362*, 2026.

[24] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2019.

[25] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.